vllm-sr/Vela-2.0-0.3B

Like 37 Follow vLLM Semantic Router 410

Zero-Shot Classification · ONNX · Safetensors · 7 datasets · 17 languages · vela2-unified · decision-model · system-one · modernbert · semantic-router · vela · token-classification · pii-detection · hallucination-detection · content-safety · multilingual · long-context · custom_code · License: apache-2.0

→ Copy to bucket NEW



Docs | Blog | GitHub | Vela 2.0 collection

Vela 2.0 0.3B

Open Foundation Routing Models

Routing decisions. Safety checks. Precise text spans.

Vela 2.0 brings routing questions and span-level decisions into one model interface. Supply your options, labels and rubrics at request time; ask about the request, its context and the answer together.

Downloads last month
2,226



Safetensors

Model size	0.3B params
Tensor type	F32

↗ Files info

Inference Providers NEW

Zero-Shot Classification

This model isn't deployed by any Inference Provider.

1 Ask for provider support

Model tree for vllm-sr/Vela-2.0-…

Base model jhu-clsp/mmBERT-base
Finetuned vllm-sr/Vela-1.0-E…
Finetuned vllm-sr/Decisio…
Finetuned (3) this model
Quantizations 1 model

The 0.3B is the family's compact multilingual encoder, with an 8,192-token input budget and Torch or ONNX inference on CPU or GPU.

1. **Route with your own criteria.** Choose a route, check a policy condition, assign a score or select multiple labels.
2. **Locate the text behind a signal.** Return spans of personal information and unsupported claims with character offsets and probabilities.
3. **Ask across the whole interaction.** Use typed request, context and answer parts in one request, through Python or a SystemOne-compatible HTTP API.

Model specification	Vela 2.0 0.3B
Backbone	22-layer bidirectional ModernBERT encoder
Hidden width	768
Input budget	8,192 tokens, including schema and state
Questions	Choice, Yes/no (Noul), Score, Span, Set
Inference	Torch fp32; ONNX fp32 or fp16 encoder with fp32 heads
Deployment	CPU or GPU; Python API and HTTP server

Quickstart

Ask for personal information, unsupported answer text and a route in one request:

Datasets used to train `vllm-sr/Ve…`

`nvidia/Aegis-AI-Content-Saf…`
Viewer • • 33.4k • 14.6k • 11

`microsoft/llmail-inject-cha…`
Viewer • • 462k • 4.31k • 35

`nvidia/Nemotron-Safety-Guar…`
Viewer • • 515k • 2.26k • 34

Spaces using `vllm-sr/Vela-2.0…` 3

vllm-sr/vela2-studio
tardellirs/model-pulse
emanforhelp/model-pulse

Collection including `vllm-sr/Vela…`

Vela 2.0 Collection
Towards O… • 5 items • Upd… • 30

For focused examples, see Span questions and Set questions below.

```
pip install torch "transformers>=4.57" safet
```

```
from transformers import AutoModel

m = AutoModel.from_pretrained("vllm-sr/Vela-
# Runs on CPU by default; optionally use m =
PII_LABELS = m.vela2_engine.cal["pii_schema"]

result = m.system_one(
    state={"request": "Hi, I'm Tom Baker (to
           "source": "For adults, the maximu
           "answer": "Adults can take up to
    questions={
        "pii": {"type": "span", "instructions
                "over": "request"},
        "halu": {"type": "span", "instructio
                 "criteria": {"unsupported":
        "domain": {"type": "choice", "instru
                   "criteria": {"health": "m
                                "math": "ari
                                "other": "a
    })
```

Selected fields from the recorded Torch CPU response, with probabilities rounded to three decimals:

```
{
  "answers": {
    "pii": {"type": "noul", "noul": 1.0},
    "halu": {"type": "noul", "noul": 0.984},
    "domain": {"type": "choice", "choice": "
  },
  "spans": {
    "pii": [{"label": "PERSON", "start": 8,
```

```
    "halu": [{"label": "unsupported", "start
  }
}
```

Offsets are Unicode code-point offsets into the targeted field. Full response, HTTP, SDK and ONNX examples.

Span questions: locate text

Use `"type": "span"` to extract text with labels and character offsets. Set `criteria` to a label-to-description mapping and `over` to the state field to inspect. This PII example reuses `m` and the trained `PII_LABELS` from above:

```
span_result = m.system_one(
    state={"request": "Hi, I'm Tom Baker (tor
    questions={
        "pii": {
            "type": "span",
            "instructions": "Which spans are
            "criteria": PII_LABELS,
            "over": "request",
        }
    },
)
print(span_result["spans"]["pii"])
```

Each returned span includes `label`, `text`, `start`, `end` and `probability`. Offsets use Unicode code points, with `start` inclusive and `end` exclusive, in the selected field. The first example also shows hallucination detection: use the `unsupported` label, target `over="answer"`, and supply grounding text in `state["source"]`.

Set questions: select multiple labels

Use `"type": "set"` when zero, one or several labels can apply. Supply your labels and descriptions in `criteria`; each label is scored independently. This example reuses `m` from above:

```
set_result = m.system_one(
    state="My card was charged twice and the
    questions={
        "issues": {
            "type": "set",
            "instructions": "Which issues do
            "criteria": {
                "billing": "payments, charge
                "shipping": "delivery of an
                "login": "signing in, passwo
            },
        }
    },
)
print(set_result["sets"]["issues"]["selected
print(set_result["sets"]["issues"]["probabil
```

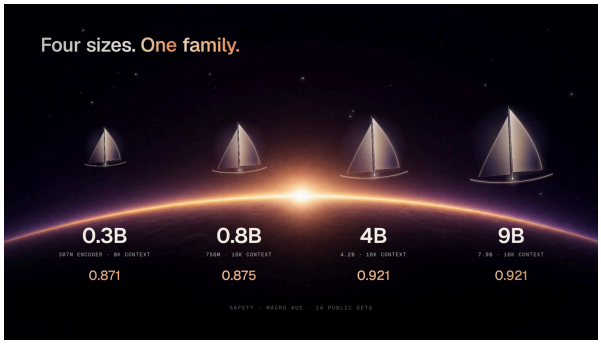
Read the selected labels and all per-label probabilities from `sets["issues"]`. Probabilities do not need to sum to one. Labels are selected when their probability exceeds the question's threshold; omit `threshold` to use shipped calibration, or add it to the question to set your own cutoff. The applied value is returned in `set_result["thresholds"]["issues"]`.

Questions and outputs

Question	Use it for	Output
Choice	Route a request or pick one of 2–255 supplied options.	Selected key and distribution
Yes/no (Noul)	Check a condition against the state.	P(yes)
Score	Rate against 2–10 ordered levels.	Expected level and distribution
Span	Locate personal information or claims unsupported by the source.	Labels, text, offsets and probabilities
Set	Select any number of supplied labels.	Selected labels and per-label probabilities

All five types share one interface. Span and Set preserve SystemOne compatibility through additional response fields and Noul views. Request and response reference.

Results

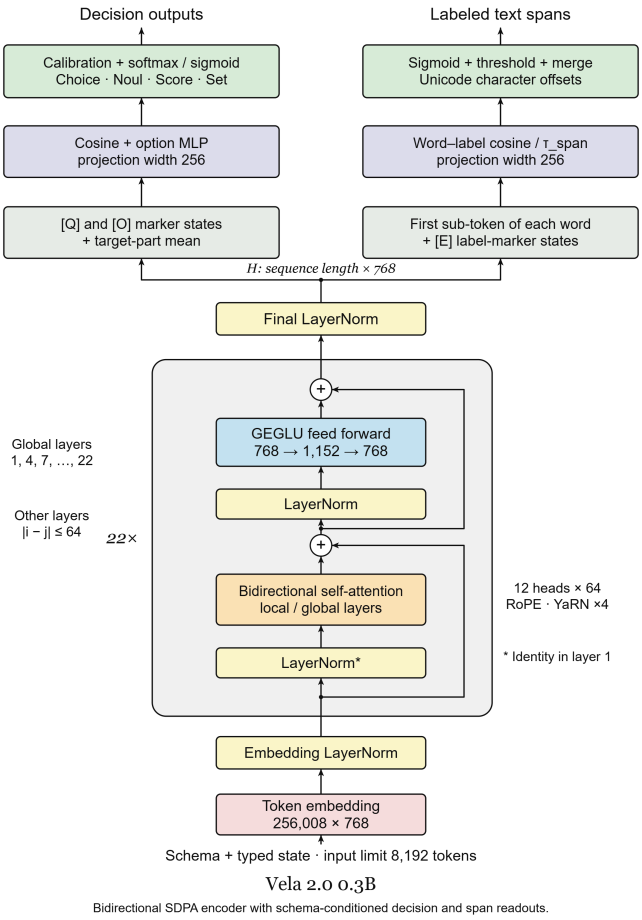


Task	Vela 2.0 0.3B	Reference
Router safety, macro AUC over 14 sets	**0.871**	GLiNER2.5-Decide 0.704
Prompt attacks, unseen families, AUC	**0.882**	Vela 1.0 Guard 0.792
Short PII, exact micro-F1	**0.995**	Vela 1.0 PII 0.976

- **Selection:** checkpoint selection used dev splits; final release selection also considered test results.
- **Safety:** these task families were part of Vela's training, unlike Decide's. This comparison measures router safety performance, rather than zero-shot generalisation.

Selected results above use identical rows for each task comparison. Full results and evaluation protocol include every benchmark, confidence intervals and the separate shipped, test-blind and research PII calibration paths.

Architecture



A 22-layer bidirectional ModernBERT encoder with schema-conditioned decision and word–label span readouts. The first attention pre-norm is Identity; this diagram shows the SDPA path. Editable SVG.

▶ Operator and readout diagrams

The backbone reads schema and typed state bidirectionally. Decision heads combine option cosine scores with an option MLP; the span head compares word and label states. Model, calibration and training details.

Reference

- Usage: Python, HTTP, SDK, Span/Set, typed parts, shortcuts and ONNX.
- Evaluation: full tables, scoring paths, references and selection disclosures.

- Training and model details: data, training recipe, input schema, calibration and repository files.
- Export parity: Torch/research-scorer and ONNX comparisons on the measured rows.

Credit and citation

Vela 2.0 is led by KR Labs and vLLM Semantic Router.

Read the Vela 2.0 technical overview.

```
@misc{vela2_unified_2026,
  title        = {Vela 2.0: Towards Open Fou
  author       = {{KR Labs} and {vLLM Semant
  year         = {2026},
  note         = {Blog post. Model: vllm-sr/
  howpublished = {\url{https://vllm-sr.ai/bl
}
```

Licence

Apache-2.0 for this model's weights, code and is derived from Decision-1.0-Kai-0.6B (Apache-2.0) and Vela-1.0-Encoder-307M (MIT, from mmBERT); their licences and notices are passed on in `LICENSE`, `NOTICE`, `NOTICE_SOURCES.json` and `LICENSES/`. The tokenizer carries the Gemma Terms of Use: DISTRIBUTION_TERMS.md · License scope. Changes against Kai and Vela, including the eight added tokenizer entries: MODIFICATIONS.md. Training data keep their own licences, some of them CC-BY-SA share-alike (see `NOTICE_SOURCES.json`).

System theme TOS Privacy About Careers Models Datasets Spaces Pricing Docs

Explore the Vela model collection